

DEV_IMPL_GUIDE - FOUNDATION_CAMUNDA

1. Purpose

This guide explains how to implement the Camunda foundation.

Use it with FOUNDATION_CAMUNDA.md.

This is a provisioning/runtime-setup guide. It is intentionally written as platform setup steps. Business BPMN execution flows are documented in the relevant workflow implementation guides.

2. What You Are Building

You are deploying Camunda as the workflow runtime:

- Camunda app nodes.
- Shared Camunda PostgreSQL database.
- External task API.
- Message correlation.
- User tasks.
- Timers.
- Operational monitoring.

3. Step 1 - Provision Runtime

Medium starting point:

Component	Count
Camunda app nodes	2
Camunda DB	Foundation DB Cluster C
Load balancer	1 internal endpoint

Camunda nodes are stateless. Workflow state is in PostgreSQL.

4. Step 2 - Configure Database

Create Camunda DB and user:

```
CREATE DATABASE sophix_camunda;  
CREATE USER sophix_camunda_app WITH PASSWORD '<secret>';  
GRANT ALL PRIVILEGES ON DATABASE sophix_camunda TO sophix_camunda_app;
```

Use Cluster C from FOUNDATION_DB.

5. Step 3 - Configure Camunda App

Configure:

- JDBC URL.
- DB credentials from secrets manager.
- Internal base URL.
- Metrics.
- Health endpoint.
- Admin access through VPN/private network only.

Verification:

```
curl -k https://camunda.internal.sophix/engine-rest/engine
```

6. Step 4 - Deploy BPMN

Example deployment:

```
POST /engine-rest/deployment/create
Content-Type: multipart/form-data
```

Deployment rules:

- Use stable process keys.
- Version BPMN artifacts.
- Deploy through CI/CD, not manual UI upload in production.
- Keep process variables small.

7. Step 5 - Implement External Task Workers

Worker loop:

1. fetchAndLock.
2. Validate variables.
3. Call module API.
4. complete, failure, or bpmnError.

Worker commands may be Laravel Artisan commands, Java processes, or container entrypoints.

They should run continuously under Kubernetes/ECS/systemd/Supervisor.

8. Step 6 - Message Correlation

Use message correlation for:

- Payment confirmed.
- Fulfillment completed/failed.
- Customer reply.
- External cancel.

Rule:

- If message arrives before process is waiting, store it and retry.
- Do not drop unmatched messages.

9. Step 7 - User Tasks

Rules:

- Users complete tasks through module APIs.
- Module API checks FOUNDATION_AUTH.
- Do not expose raw Camunda UI/API to normal users.
- Store task completion payload in module audit/history.

10. Step 8 - Monitoring

Alert on:

Metric	Alert
Camunda node down	One node down warning, all down critical.
Camunda DB unavailable	Critical.
External task backlog	Above SLA threshold.
Incidents	Any sustained count.

Metric	Alert
Failed message correlations	Above zero sustained.
Timer/job executor failures	Above zero sustained.

11. AWS Option

If using AWS:

- Run Camunda on ECS/EKS or EC2 ASG.
- Use RDS PostgreSQL Cluster C.
- Use private ALB.
- Use CloudWatch logs/metrics.
- Store secrets in Secrets Manager or SSM.

12. Tests

Test:

- Deploy BPMN.
- Start process.
- Fetch and complete external task.
- Correlate message.
- Complete user task through module API.
- Stop one Camunda node and continue on the other.

13. Common Mistakes

- Doing business logic inside BPMN scripts.
- Storing large payloads in process variables.
- Calling users directly to Camunda API.
- No unmatched-message handling.
- No incident monitoring.